



Preface

This book is a guide to modern Python developer tools—the programs that help you perform tasks such as:

- Managing Python installations on your system
- Installing third-party packages for your current project
- Building a Python package for distribution on a package repository
- Running a test suite repeatedly across multiple environments
- Linting and type checking your code to catch bugs

You don't strictly need these tools to write Python software. Fire up your system's Python interpreter and get an interactive prompt. Save your Python code as a script for later. Why use anything beyond an editor and a shell?

This is not a rhetorical question. Every tool you add to your development workflow should have a clear purpose and bring benefits that outweigh the costs of using it. Generally, the benefits of development tooling become manifest when you need to make development sustainable *over time*. At some point, publishing your module on the Python Package Index will be easier than emailing it to your users.

Somewhere along the journey from writing one-off scripts to distributing and maintaining packages, challenges pop up:

- Supporting multiple versions of Python on multiple operating systems
- Keeping dependencies up-to-date and scanning them for vulnerabilities
- Keeping the code base readable and consistent

- Interacting with bug reports and external contributions from the community
- Keeping test coverage high to reduce the defect rate in code changes
- Automating repeated tasks to reduce friction and avoid surprises

This book will show you how developer tooling can help with such challenges. The tools described here greatly benefit the code quality, security, and maintainability of Python projects.

But tooling also adds complexity and overhead. This book strives to minimize that by forging tools into an easy-to-use toolchain and by automating workflows reliably and repeatably—whether they execute locally on a developer machine, or on a continuous integration server across a range of platforms and environments. As much as possible, you should be able to focus your attention on writing software, with your toolchain working in the background.

Laziness has been called “a programmer’s greatest virtue,”¹ and this saying applies to development tooling, too: keep your workflow simple, and don’t adopt tools for their own sake. At the same time, good programmers are also curious. Give the tools in this book a try to see what value they may bring to your projects.

The Python ecosystem constantly evolves: what we consider modern today inevitably yields to the *hypermodern* ways of tomorrow.² This book attempts to raise awareness for recent tools, standards, and best practices—not for newness’ sake, but because they improve your productivity and developer experience. Understanding the whys and the hows of these tools and workflows—as well as their tradeoffs—will prepare you for when the next generation of Python tooling appears at the horizon.

Who Should Read This Book?

If you’re one of these people, you’ll benefit from reading this book:

- You’re proficient with Python, but you’re not sure how to create a package.

- You’ve been doing this for years—setuptools, virtualenv, and pip are your friends. You’re curious about recent developments in tooling and what they bring to the table.
- You maintain mission-critical code that runs in production. But there must be a better way to do all of this. You want to learn about state-of-the-art tools and evolving best practices.
- You want to be more productive as a Python developer.
- You’re an open source maintainer looking for a robust and modern project infrastructure.
- You’re using a bunch of Python tools in your projects, but it’s hard to see how everything fits together. You want to reduce the friction that comes with all this tooling.
- “Things just keep breaking—why doesn’t Python find my module now? Why can’t I import the package I just installed?”

This book assumes that you have a basic knowledge of the Python programming language. The only tooling you need to be familiar with is the Python interpreter, an editor or IDE, and the command line of your operating system.

How This Book Is Organized

This book is arranged in three parts:

Part I, “Working with Python”

- *Chapter 1, “Installing Python”*, teaches you how to manage Python installations over time across different platforms. The chapter also introduces the Python Launchers for Windows and Unix—you’ll use them throughout the book.
- *Chapter 2, “Python Environments”*, zooms into a Python installation and discusses how your code interacts with it. You’ll also learn about tools that help you work efficiently with virtual environments.

Part II, “Python Projects”

- [Chapter 3, “Python Packages”](#), teaches you how to set up your project as a Python package, and how to build and publish packaging artifacts. The chapter also introduces the example application used throughout the book.
- [Chapter 4, “Dependency Management”](#), describes how to add third-party packages to a Python project and how to keep track of your project dependencies over time.
- [Chapter 5, “Managing Projects with Poetry”](#), teaches you how to work with Python projects using Poetry. Poetry lets you manage environments, dependencies, and packaging at a higher level.

Part III, “Testing and Static Analysis”

- [Chapter 6, “Testing with pytest”](#), discusses how to test a Python project and how to work efficiently with the pytest framework and its ecosystem.
- [Chapter 7, “Measuring Coverage with Coverage.py”](#), teaches you how to discover untested code by measuring the code coverage of your test suite.
- [Chapter 8, “Automation with Nox”](#), introduces the Nox automation framework. You’ll use it to run tests across Python environments and, more generally, to automate checks and other developer tasks in your projects.
- [Chapter 9, “Linting with Ruff and pre-commit”](#), shows you how to find and fix likely bugs, and how to format your code, with

Ruff. You'll also learn about pre-commit, a cross-language linter framework with Git integration.

- [Chapter 10, “Using Types for Safety and Inspection”](#), teaches you how to verify type safety with static and runtime type checkers, and how to inspect types at runtime to perform actual magic (terms and conditions apply).

References and Further Reading

Your first point of reference outside this book should be the official documentation of each tool. Besides those, many interesting packaging-related discussions take place on the [Python Discourse](#). Discussions in the Packaging category often shape the future of the Python packaging and tooling ecosystem in the form of packaging standards using the [Python Enhancement Proposal \(PEP\) process](#). Finally, the [Python Packaging Authority \(PyPA\)](#) is a working group that maintains a core set of software projects used in Python packaging. Its website tracks the list of currently active interoperability standards governing Python packaging. The PyPA also publishes the [Python Packaging User Guide](#).

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

WARNING

This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://oreil.ly/hmpt-code>.

If you have a technical question or a problem using the code examples, please email bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting

example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Hypermodern Python Tooling* by Claudio Jolowicz (O’Reilly). Copyright 2024 Claudio Jolowicz, 978-1-098-13958-2.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O’Reilly Online Learning

NOTE

For more than 40 years, [O’Reilly Media](#) has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O’Reilly’s online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O’Reilly and 200+ other publishers. For more information, visit <https://oreilly-com.ezproxy.sfpl.org>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

- O’Reilly Media, Inc.
- 1005 Gravenstein Highway North

- Sebastopol, CA 95472
- 800-889-8969 (in the United States or Canada)
- 707-827-7019 (international or local)
- 707-829-0104 (fax)
- support@oreilly.com
- <https://oreilly-com.ezproxy.sfpl.org/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at

<https://oreil.ly/hypermodern-python-tooling>.

For news and information about our books and courses, visit

<https://oreilly-com.ezproxy.sfpl.org>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>

Watch us on YouTube: <https://youtube.com/oreillymedia>

Acknowledgments

This book covers many open source Python projects. I am very grateful to their authors and maintainers, most of whom work on them in their free time, often over many years. In particular, I would like to acknowledge the unsung heroes of the PyPA, whose work on packaging standards lets the ecosystem evolve toward better tooling. Special thanks to Thea Flowers for writing Nox and building a welcoming community.

Before this book, there was the Hypermodern Python article series. I would like to thank Brian Okken, Michael Kennedy, and Paul Everitt for spreading the word, and Brian for giving me the courage to turn it into a book.

For their deep insights and opinionated feedback, I am indebted to my reviewers Pat Viafore, Jürgen Gmach, Hynek Schlawack, William Jamir

Silva, Ganesh Hark, and Karandeep Johar. This book wouldn't be the same without them. I take responsibility for any errors that remain.

Making a book takes a village. I would like to thank my editors Zan McQuade, Brian Guerin, Sarah Grey, Greg Hyman, and Emily Wydeven, as well as all the team at O'Reilly. My special thanks go out to Sarah for keeping me on track and improving my writing during this journey. For giving me time to work on this book, I thank Jakub Borys, my manager at Cloudflare. Thank you, Gabriela Jolowicz, for creating such beautiful and compelling chapter vignettes featuring past visions of the future.

This book is dedicated to Marianna, the love of my life. Without her support, encouragement, and inspiration, I couldn't have written this book.

1 Larry Wall, *[Programming Perl](#)* (Sebastopol: O'Reilly, 1991).

2 The title of this book is inspired by *Die hypermoderne Schachpartie* (The hyper-modern chess game), written by Savielly Tartakower in 1924. It surveys the revolution that was taking place in chess theory during his time.